

K8s-06

1. Create a Config Map from a directory containing multiple files and inject the variables into a pod as environment variables.

- Create 1st configuration files

```
root@Master:~/k8s-06# vi app.properties
root@Master:~/k8s-06#
```

```
APP_NAME=InventoryApp
APP_ENV=Development
```

- Create 2nd configuration file

```
root@Master:~/k8s-06# vi db.properties
root@Master:~/k8s-06#
```

```
DB_HOST=mysql
DB_PORT=3306
```

- Verify the files

```
root@Master:~/k8s-06# ls -l
total 12
-rw-r--r-- 1 root root 42 Jul  1 10:49 1
-rw-r--r-- 1 root root 42 Jul  1 10:49 app.properties
-rw-r--r-- 1 root root 27 Jul  1 10:52 db.properties
```

- Create a Config Map from the directory and verify it and describe

```
root@Master:~/k8s-06# kubectl create configmap app-config --from-file=.
configmap/app-config created
root@Master:~/k8s-06# kubectl get configmap
NAME          DATA  AGE
app-config    3      24s
kube-root-ca.crt 1      7d
root@Master:~/k8s-06# kubectl describe configmap app-config
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>
```

- Create a Pod that uses the Config Map as environment variables

```
root@Master:~/k8s-06# vi configmap-env-pod.yaml
root@Master:~/k8s-06#
```

```

apiVersion: v1
kind: Pod
metadata:
  name: configmap-env-pod
spec:
  containers:
  - name: nginx
    image: nginx
    env:
    - name: APP_NAME
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: app.properties
    - name: DB_CONFIG
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: db.properties

```

- Apply and verify it as Config Map injected as environment variables.

```

root@Master:~/k8s-06# kubectl apply -f configmap-env-pod.yaml
pod/configmap-env-pod created
root@Master:~/k8s-06# kubectl get pod configmap-env-pod
NAME                READY   STATUS    RESTARTS   AGE
configmap-env-pod   1/1     Running   0           8s
root@Master:~/k8s-06# kubectl exec configmap-env-pod -- env | grep -E "APP_NAME|DB_CONFIG"
APP_NAME=APP_NAME=InventoryApp
DB_CONFIG=DB_HOST=mysql

```

2. Create a Config Map from a file and mount it as a volume inside a pod, ensuring the configuration data is available as files.

- Create a configuration file

```

root@Master:~/k8s-06# vi nginx.conf
root@Master:~/k8s-06# 

```

```

server {
    listen 80;
    server_name localhost;
}
~

```

- Create a Config Map from the file and verify it

```

root@Master:~/k8s-06# kubectl create configmap nginx-config --from-file=nginx.conf
configmap/nginx-config created
root@Master:~/k8s-06# kubectl describe configmap nginx-config
Name:         nginx-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
nginx.conf:
-----
server {
    listen 80;
    server_name localhost;
}

BinaryData
====

Events:  <none>
root@Master:~/k8s-06# 

```

- Create the Pod

```
root@Master:~/k8s-06# vi configmap-volume-pod.yaml
root@Master:~/k8s-06#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-volume-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: config-volume
      mountPath: /etc/config
      readOnly: true
  volumes:
  - name: config-volume
    configMap:
      name: nginx-confi
```

- Apply and verify it

```
root@Master:~/k8s-06# kubectl apply -f configmap-volume-pod.yaml
pod/configmap-volume-pod created
root@Master:~/k8s-06# kubectl get pod configmap-volume-pod
NAME                READY   STATUS    RESTARTS   AGE
configmap-volume-pod 1/1     Running   0           9s
```

- List the mounted files and display the file:

```
root@Master:~/k8s-06# kubectl exec configmap-volume-pod -- ls /etc/config
nginx.conf
root@Master:~/k8s-06# kubectl exec configmap-volume-pod -- cat /etc/config/nginx.conf
server {
    listen 80;
    server_name localhost;
}
root@Master:~/k8s-06#
```

3. Create a Secret with sensitive information (username and password) and inject it into a pod as environment variables.

- Create the Secret

```
root@Master:~/k8s-06# kubectl create secret generic app-secret \
  --from-literal=username=admin \
  --from-literal=password=Admin@123
secret/app-secret created
```

- Verify and describe it

```
root@Master:~/k8s-06# kubectl get secrets
NAME                TYPE      DATA   AGE
app-secret           Opaque    2        10s
root@Master:~/k8s-06# kubectl describe secret app-secret
Name:          app-secret
Namespace:     default
Labels:        <none>
Annotations:   <none>

Type: Opaque

Data
====
password:  9 bytes
username:  5 bytes
```

- Create secret-env-pod. Yaml

```
root@Master:~/k8s-06# vi secret-env-pod.yaml
root@Master:~/k8s-06#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
  - name: nginx
    image: nginx
    env:
    - name: USERNAME
      valueFrom:
        secretKeyRef:
          name: app-secret
          key: username
    - name: PASSWORD
      valueFrom:
        secretKeyRef:
          name: app-secret
          key: password
```

- Apply pod and verify the environment variables

```
root@Master:~/k8s-06# kubectl apply -f secret-env-pod.yaml
pod/secret-env-pod created
root@Master:~/k8s-06# kubectl get pod secret-env-pod
NAME          READY   STATUS    RESTARTS   AGE
secret-env-pod 1/1     Running   0           9s
root@Master:~/k8s-06# kubectl exec secret-env-pod -- env | grep -E "USERNAME|PASSWORD"
PASSWORD=Admin@123
USERNAME=admin
root@Master:~/k8s-06#
```

4. Create a Secret using a YAML file, mount it as a volume in a pod, and verify the specific Secret values are available as files.

- Encode the Secret Values
- Generate Base64 values and password

```
root@Master:~/k8s-06# echo -n "admin" | base64
YWRtaW4=
root@Master:~/k8s-06# echo -n "Admin@123" | base64
QWRtaW5AMTIz
root@Master:~/k8s-06#
```

- Create secret.yaml

```
root@Master:~/k8s-06# vi secret.yaml
root@Master:~/k8s-06#
```

```

apiVersion: v1
kind: Secret
metadata:
  name: yaml-secret
type: Opaque
data:
  username: YWRtaW4=
  password: QWRtaW5AMTI=

```

- Apply the Secret and Verify

```

root@Master:~/k8s-06# kubectl apply -f secret.yaml
secret/yaml-secret created
root@Master:~/k8s-06# kubectl describe secret yaml-secret
Name:          yaml-secret
Namespace:     default
Labels:        <none>
Annotations:   <none>

Type:  Opaque

Data
====
password:  9 bytes
username:  5 bytes
root@Master:~/k8s-06#

```

- Create the pod

```

root@Master:~/k8s-06# vi secret-volume-pod.yaml
root@Master:~/k8s-06#

```

```

apiVersion: v1
kind: Pod
metadata:
  name: secret-volume-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secret
      readOnly: true

  volumes:
  - name: secret-volume
    secret:
      secretName: yaml-secret

```

- Apply and verify

```

root@Master:~/k8s-06# kubectl apply -f secret-volume-pod.yaml
pod/secret-volume-pod created
root@Master:~/k8s-06# kubectl get pod secret-volume-pod
NAME                READY   STATUS    RESTARTS   AGE
secret-volume-pod   1/1     Running   0           8s
root@Master:~/k8s-06# kubectl exec secret-volume-pod -- ls /etc/secret
password
username
root@Master:~/k8s-06# kubectl exec secret-volume-pod -- cat /etc/secret/username
adminroot@Master:~/k8skubectl exec secret-volume-pod -- cat /etc/secret/passwordsword
Admin@123root@Master:~/k8s-06#

```

5. Inject a Config Map as environment variables and a Secret as files into the same pod, ensuring both are accessible within the pod.

- Create combined-pod. Yaml

```
root@Master:~/k8s-06# vi combined-pod.yaml
root@Master:~/k8s-06#
```

```
apiVersion: v1
kind: Pod
metadata:
  name: combined-pod
spec:
  containers:
  - name: nginx
    image: nginx

    env:
    - name: APP_CONFIG
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: app.properties

    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secret
      readOnly: true

  volumes:
  - name: secret-volume
    secret:
      secretName: yaml-secret
```

- Apply the Pod and verify

```
root@Master:~/k8s-06# kubectl apply -f combined-pod.yaml
pod/combined-pod created
root@Master:~/k8s-06# kubectl get pod combined-pod
NAME          READY   STATUS    RESTARTS   AGE
combined-pod  1/1     Running   0           7s
root@Master:~/k8s-06#
```

- Verify the Config Map Environment Variable

```
combined-pod 1/1 Running 0 7s
root@Master:~/k8s-06# kubectl exec combined-pod -- env | grep APP_CONFIG
APP_CONFIG=APP_NAME=InventoryApp
```

- Verify the Secret Files

```
root@Master:~/k8s-06# kubectl exec combined-pod -- ls /etc/secret
password
username
root@Master:~/k8s-06# kubectl exec combined-pod -- cat /etc/secret/username
adminroot@Master:~/k8skubectl exec combined-pod -- cat /etc/secret/passwordsword
Admin@123root@Master:~/k8s-06#
```